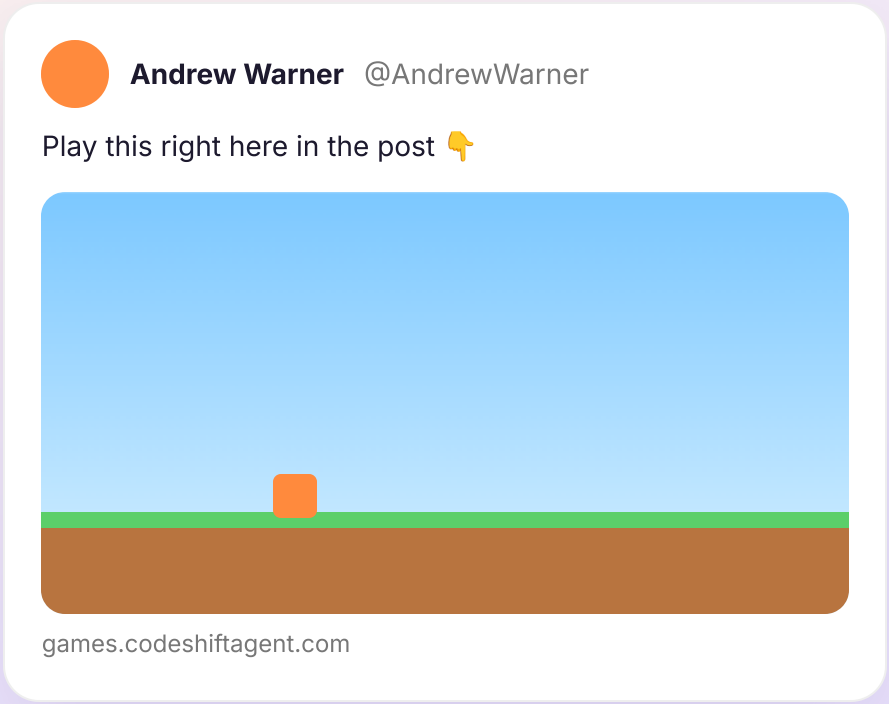
A FIELD GUIDE · 2026

Put a playable app inside a tweet.

How X player cards let a game or a form run right inside a post, and a copy-paste prompt your AI coding agent can use to build one.



By **Andrew Warner**

The trick in one paragraph

01

When you paste a link into a post, X's crawler (it identifies itself as `Twitterbot`) fetches the page and reads its meta tags. If the page says `twitter:card = player` and points to an HTTPS page with `twitter:player`, X can show your page in an iframe inside the post. That page can be anything a browser can run: a video, a game, a form.

The tags

`twitter:card`	`player`
`twitter:player`	HTTPS URL of the page that runs inside the iframe
`twitter:player:width / height`	The iframe size in pixels, e.g. 640 × 480
`twitter:image`	Preview image shown before the iframe loads, and where players aren't supported. Match the player's aspect ratio.
`twitter:title / description`	Text shown with the card
`og:*`	Open Graph equivalents, for other sites and as a fallback

A real example

Pieter Levels (@levelsio) used this to boot Windows XP inside a post. His share link's page has these tags:

```
<meta name="twitter:card" content="player">
<meta name="twitter:player" content="https://pieter.com/?embed=1">
<meta name="twitter:player:width" content="640">
<meta name="twitter:player:height" content="480">
<meta name="twitter:image" content="https://pieter.com/x/card-xp.jpg">
```

Two details worth copying: the iframe points to a stripped-down, screen-only version of the app (`?embed=1`), and humans who open the share link directly are redirected to the full site with a small script.

> Use `name="twitter:…"` for X's tags. Open Graph uses `property="og:…"`. Mixing them up is a common reason cards don't show.

What we learned shipping it

02

I built a tiny platformer, *Pip's Run*, and this signup form, and ran both inside posts from `games.codeshiftagent.com`. Here's what mattered.

1. It loads on click

X shows the preview image first. The player iframe loads when someone taps it, so your preview image is your "press start" screen. Make it look like the app.

2. Mouse and taps work. Keyboard focus is unreliable.

Our first version used arrow keys, and inside X the keys did nothing: the keystrokes stayed with X's page instead of reaching the iframe. Clicks and taps always reached it. What fixed it:

- Never call `preventDefault()` on `pointerdown`. It stops the browser from moving focus into the iframe.
- On any tap or click, call `window.focus()` and focus your canvas or input.
- Design for mouse and touch first: on-screen buttons, "run toward the pointer", click to jump.
- For forms: focus the input on tap, and offer an "open in new tab" fallback.

3. Don't block framing

Serve over HTTPS and don't send `X-Frame-Options` or a restrictive CSP `frame-ancestors`. Our first deploy whitelisted only x.com and twitter.com, which can break if X frames the player from another origin. Pieter sends no framing headers at all, and that is what we use now.

4. Caching

X caches card metadata for a while; guides commonly cite up to about 7 days. If you change the tags or image, share the URL with a new query string like `?v=3`. If you only change the player page itself, no new URL is needed: the iframe loads your live page every time.

5. The approval caveat

> Historically, X required player-card domains to be approved, and the old Card Validator no longer shows previews. Third-party guides still say approval is needed. In our case a brand-new subdomain rendered as a player card. Treat that as "worked for us," not a guarantee. Test by pasting your URL into a draft post without publishing.

The architecture

03

Three URLs, one tiny Cloudflare Worker.

```
# what you paste into the post
https://example.com/join/x       → card page: meta tags only (+ redirect for humans)
# what X loads inside the iframe
https://example.com/join/play    → the app/form, built to fit 640×480
# preview image
https://example.com/join/card.png
```

For a lead form, add a backend route

```
POST /join/api/subscribe
  1. validate email (and a hidden honeypot field)
  2. rate-limit by IP
  3. POST https://api.beehiiv.com/v2/publications/{PUB_ID}/subscriptions
       Authorization: Bearer {BEEHIIV_API_KEY}
       { "email": "...", "send_welcome_email": true,
         "utm_source": "x", "utm_medium": "tweet-card" }
  4. return { ok: true } → show thank-you + PDF link
```

Keep API keys in Worker secrets, never in the page. The form runs inside X, but it talks only to your own domain.

Checklist before you post

1. `curl -A "Twitterbot/1.0" https://example.com/join/x` shows the tags in the raw HTML (not added by JavaScript).
2. The player page loads over HTTPS with no framing-blocking headers.
3. It fits and works at 640×480 and on a phone.
4. Embed it in a test page on another domain and try it with mouse only, then touch.
5. Paste the card URL into a draft post to see the preview. Don't publish until it looks right.

For your agent: the prompt

04

Paste this into Claude Code, Codex, Cursor or any coding agent with access to your Cloudflare account.

```
Build an email signup form that runs INSIDE a post on X using a player card,
deployed as a single Cloudflare Worker.

ROUTES (all HTTPS, on my domain {DOMAIN}):
- GET /join/x      Card page. Static HTML with these tags in the <head>:
    <meta name="twitter:card" content="player">
    <meta name="twitter:title" content="{TITLE}">
    <meta name="twitter:description" content="{DESCRIPTION}">
    <meta name="twitter:image" content="https://{DOMAIN}/join/card.png">
    <meta name="twitter:player" content="https://{DOMAIN}/join/play">
    <meta name="twitter:player:width" content="640">
    <meta name="twitter:player:height" content="480">
    plus matching og:title/og:description/og:image/og:url.
    Body: a script that redirects humans to /join/play.
- GET /join/play   The form. Designed to fit 640x480 and phones. One email
    input, a hidden honeypot input named "company", a submit button,
    a thank-you state with a download link to /join/guide.pdf.
    Tapping the input must focus it (call window.focus() then input.focus()
    on pointerdown/touchstart/click). Never preventDefault on pointerdown.
    Add an "Open in new tab" link that calls window.open(...) as a fallback.
- GET /join/card.png  1200x628 or 4:3 preview image of the form.
- GET /join/guide.pdf The lead magnet.
- POST /join/api/subscribe  JSON {email, company}.
    Reject if honeypot filled or email invalid (400).
    Rate-limit per IP (CF-Connecting-IP), e.g. 5 per 10 minutes.
    If env.BEEHIIV_API_KEY or env.BEEHIIV_PUBLICATION_ID is missing,
    log it and return 503 {ok:false, message:"Signups aren't open yet"}.
    Otherwise POST https://api.beehiiv.com/v2/publications/
      ${env.BEEHIIV_PUBLICATION_ID}/subscriptions
      headers: Authorization: Bearer ${env.BEEHIIV_API_KEY},
               Content-Type: application/json
      body: {email, reactivate_existing:false, send_welcome_email:true,
             utm_source:"x", utm_medium:"tweet-card", referring_site:"x.com"}
    On 2xx return {ok:true}; otherwise log status/body, return 502
    with a friendly message.

HEADERS: do NOT send X-Frame-Options or CSP frame-ancestors.

SECRETS: wrangler secret put BEEHIIV_API_KEY
         wrangler secret put BEEHIIV_PUBLICATION_ID

TEST:
1. curl -A "Twitterbot/1.0" https://{DOMAIN}/join/x and confirm the tags.
2. Serve a local page on another origin that iframes /join/play at
   640x480; with a headless browser click the input, type an email,
   submit, and screenshot each state.
3. Submit with the honeypot filled and confirm it is rejected.
4. Paste the /join/x URL into a draft post on X (do not publish) to
   confirm the player card renders.
```

THANKS FOR READING

Now go put something fun in a post.

See it live: play *Pip's Run* at games.codeshiftagent.com/jump, and this form at games.codeshiftagent.com/join.

— **Andrew Warner**